

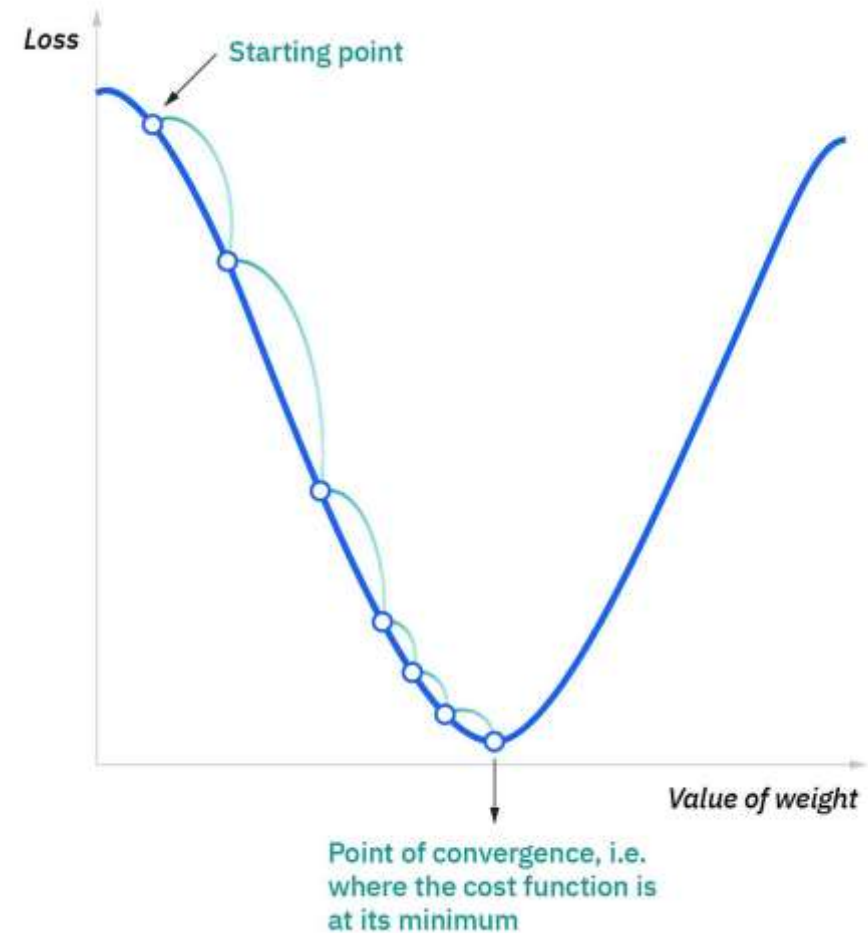


GRADIENT DESCENT

Deep Learning

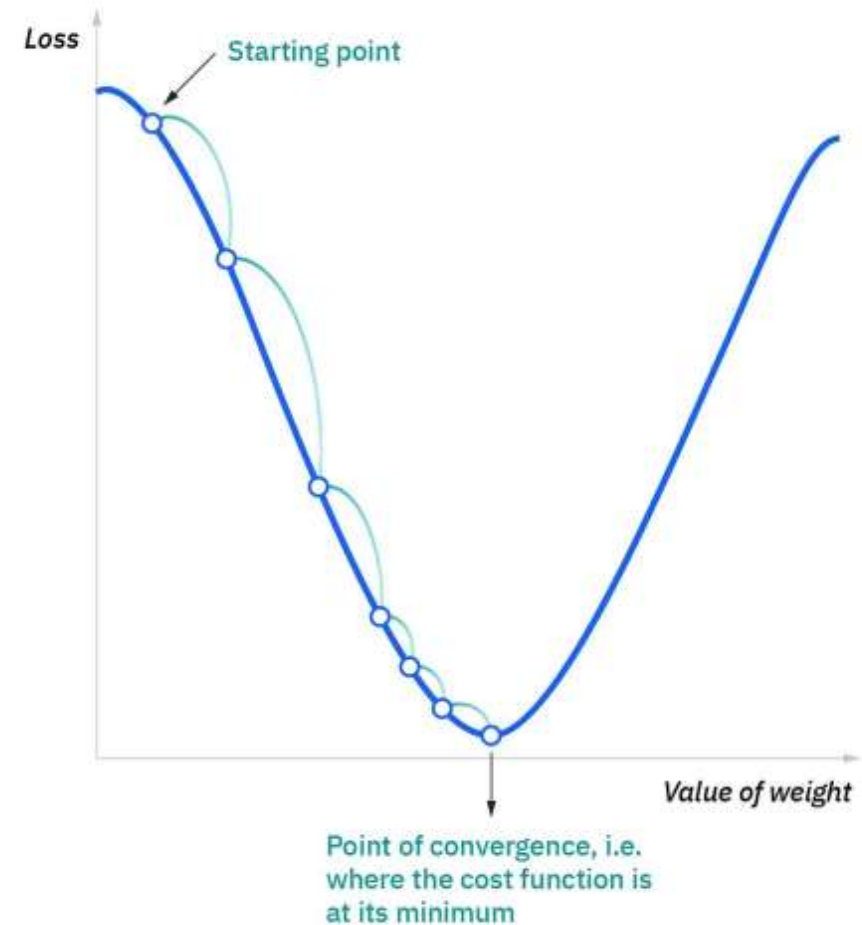
Gradient Descent

- Gradient descent is a **popular optimization** approach for training machine learning models and neural networks.
- These models evolve over time with the use of training data, and the cost function inside gradient descent especially functions as a **barometer**, assessing its correctness with **each iteration** of parameter changes.
- The model will **continue to change** its parameters until the function is near to or equal to zero, at which point it will stop.
- Works on differentiable and convex function.



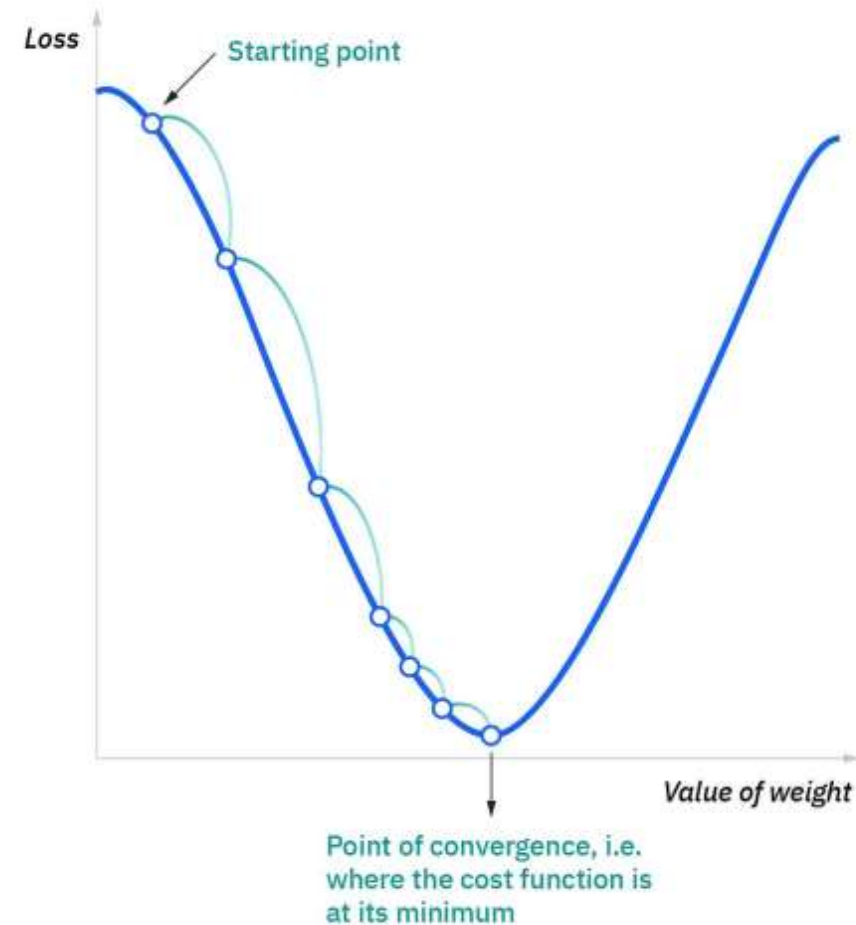
How does gradient descent work?

- The **purpose** of gradient descent, like finding the line of best fit in linear regression, is to **minimize** the cost function or the difference between expected and real values.
- This necessitates the collection of two data points: a direction and a learning rate.
- *The slope will influence parameter changes, such as weights and bias. The slope will be higher at the start, but as new parameters are created, it should steadily decrease until it hits the lowest point on the curve, known as the point of convergence.*



How does gradient descent work?

- The **purpose** of gradient descent, like finding the line of best fit in linear regression, is to **minimize** the cost function or the difference between expected and real values.
- This necessitates the collection of two data points: a direction and a learning rate.
- *The slope will influence parameter changes, such as weights and bias. The slope will be higher at the start, but as new parameters are created, it should steadily decrease until it hits the lowest point on the curve, known as the point of convergence.*

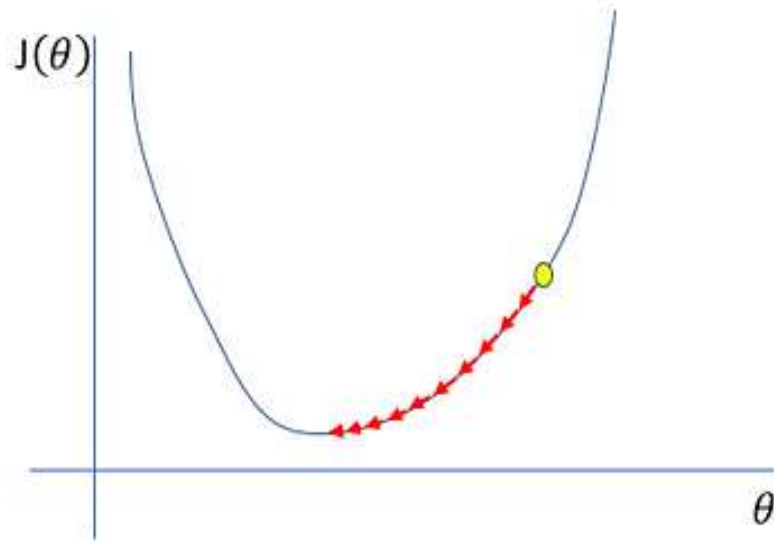


Learning rate

- Also referred to as *step size* or the *alpha*
- The size of the steps taken to reach the minimum is known as **step size**. It is usually a small number and is adjusted depending on the behavior of the cost function.
- Having a **large learning rate** will cause larger steps, but there is a chance that it could go past the minimum.
- On the other hand, a **low learning rate** will have smaller steps and be more accurate, but it will take a lot more time and calculations to reach the minimum.

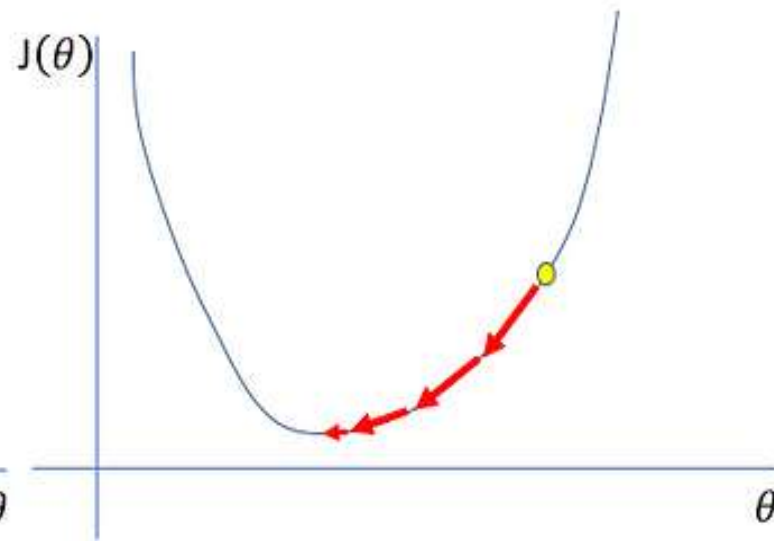
Learning rate

Too low



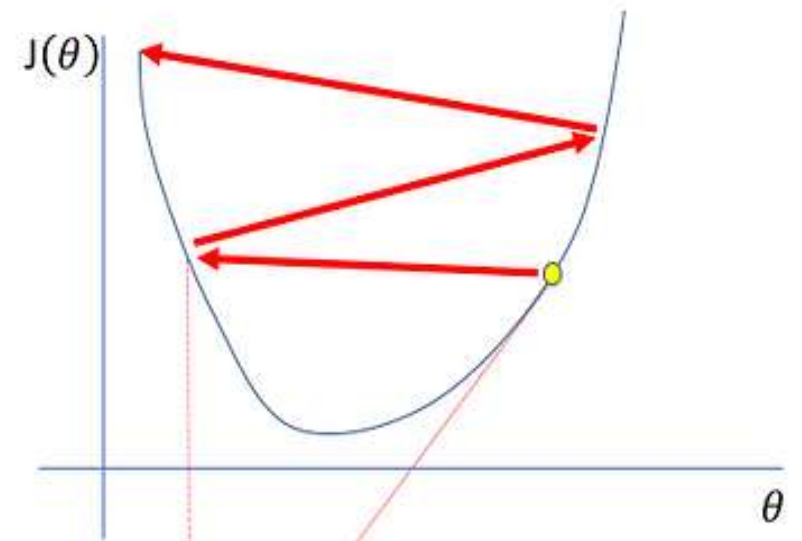
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high



Too large of a learning rate causes drastic updates which lead to divergent behaviors

Types of Gradient Descent

1. Batch Gradient Descent:

- Batch gradient descent (BGD), the entire training dataset is used to compute the gradient and update the parameters at each iteration.
- This procedure is known as the **training epoch**. In simple words, it is a greedy approach where we have to sum over all examples for each update.
- This can be slow for large datasets but may lead to a more accurate model.
- **Advantages of Batch gradient descent:**
- It produces less noise in comparison to other gradient descents.
- It produces stable gradient descent convergence.
- It is Computationally efficient as all resources are used for all training samples.

Types of Gradient Descent

2. Stochastic gradient descent

- Stochastic gradient descent (SGD) is a type of gradient descent that runs **one training example per iteration**.
- Or in other words, it processes a training epoch for each example within a dataset and updates each training example's parameters one at a time. There are some computational efficiency losses in comparison to batch gradient systems as they show frequent updates that require more detail and speed. Further, due to frequent updates, it is also treated as a noisy gradient
- **Advantages of Stochastic gradient descent:**
- In Stochastic gradient descent (SGD), **learning happens on every example**, and it consists of a few advantages over other gradient descent.
- It is easier to allocate in desired memory.
- It is relatively faster to compute than batch gradient descent.
- It is more efficient for large datasets.

Types of Gradient Descent

3. Mini Batch Gradient Descent:

- Mini Batch gradient descent is the **combination** of both batch gradient descent and stochastic gradient descent.
- It divides the training datasets into small batch sizes and then performs the updates on those batches separately. Splitting training datasets into smaller batches creates a balance to maintain the computational efficiency of batch gradient descent and speed of stochastic gradient descent.

Advantages of Mini Batch gradient descent:

- It is easier to fit in allocated memory.
- It is computationally efficient.
- It produces stable gradient descent convergence.
- Less Noisy Gradient.

Variant	Advantages	Disadvantages
Batch gradient descent	Guaranteed convergence to global optimum	Computationally expensive for large datasets, slow convergence
Stochastic gradient descent	Faster convergence, more efficient for large datasets	High variance, may not converge to global optimum
Mini-batch gradient descent	Balanced convergence speed and computational cost, efficient for large datasets	Choice of mini-batch size can be a challenge
Momentum gradient descent	Faster convergence, less likely to get stuck in local minima	May overshoot and oscillate around the optimum
Adagrad	Adaptive learning rate, efficient for sparse data	Can stop learning too early
RMSProp	Adaptive learning rate, efficient for non-stationary problems	Can stop learning too early, requires tuning of hyperparameters
Adam	Adaptive learning rate, efficient for large datasets and noisy data	Can converge to suboptimal solutions, requires tuning of hyperparameters

Formula

$$\theta = \theta - \alpha * \nabla J(\theta)$$

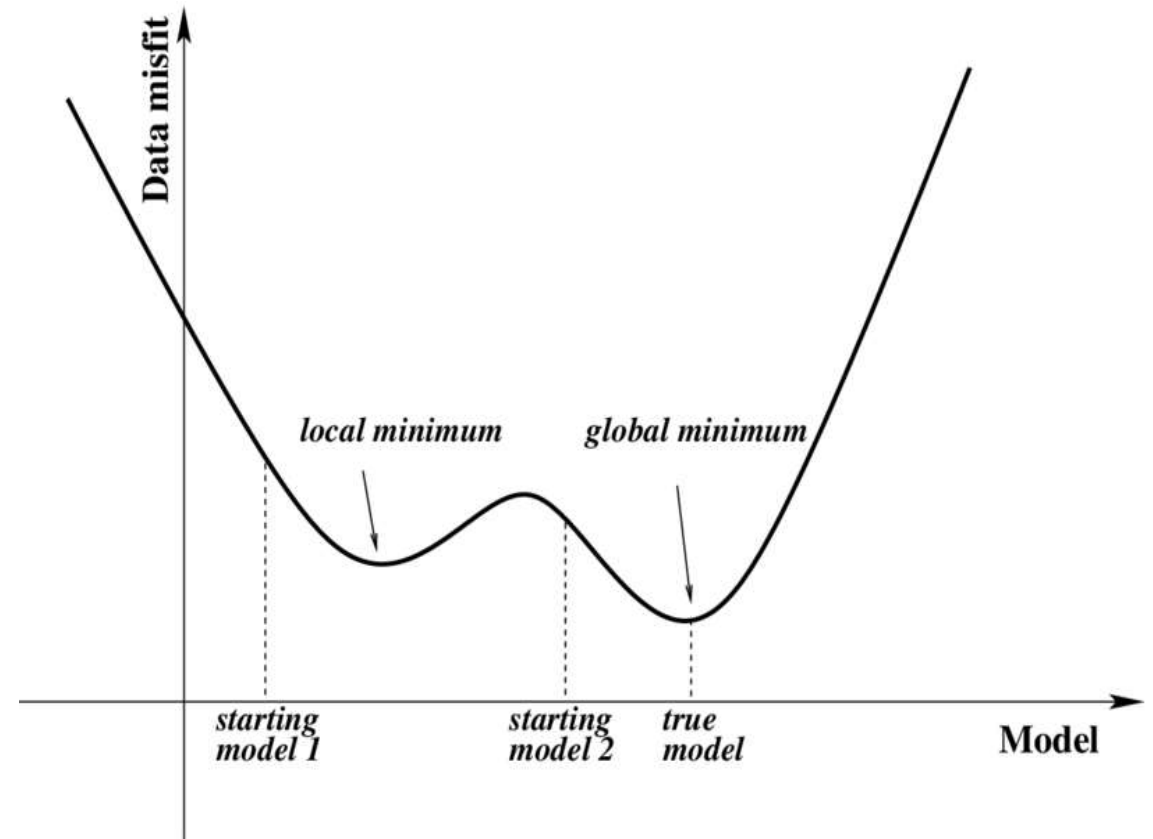
where:

- θ : the parameters to be optimized
 - α : the learning rate, which determines the step size at each iteration
 - $J(\theta)$: the cost function to be minimized
 - $\nabla J(\theta)$: the gradient of the cost function with respect to θ , which is a vector of partial derivatives
- During each iteration of gradient descent, the parameters θ are updated according to the above formula, where $\nabla J(\theta)$ is evaluated using the current values of θ .
- In each iteration, the algorithm takes a step in the direction of the steepest descent of the cost function, with a step size determined by the learning rate. The learning rate determines the size of the step taken at each iteration and needs to be carefully chosen to ensure that the algorithm converges to the optimal solution.

Challenges with Gradient Descent

Local Minima and Saddle Points:

- In nonconvex problems, gradient descent may get stuck in local minima or saddle points, where the cost function's slope approaches zero.
- Local minima appear as global minima but are not optimal, while saddle points have gradients that are zero in some directions but not others. Introducing noise can sometimes help escape these problematic points.



Challenges with Gradient Descent

Vanishing Gradients:

- During backpropagation, very small gradients can cause earlier layers in a neural network to learn too slowly, resulting in insignificant weight updates and hindered learning.

Exploding Gradients:

- Conversely, excessively large gradients can lead to instability in the model, with weights becoming too large and potentially resulting in undefined values (NaN). Dimensionality reduction can mitigate this issue by simplifying the model.

Practical use case of Gradient Descent

Linear Regression:

- In linear regression, gradient descent is used to find the optimal coefficients that minimize the sum of squared errors between the predicted and actual values.

Logistic Regression:

- In logistic regression, gradient descent is used to find the optimal parameters that minimize the cross-entropy loss function, which measures the difference between the predicted probabilities and actual labels.

Neural Networks:

- In deep learning, gradient descent is used to optimize the weights and biases of the neural network by minimizing the loss function, which measures the difference between the predicted and actual outputs.

Practical use case of Gradient Descent

Support Vector Machines (SVMs):

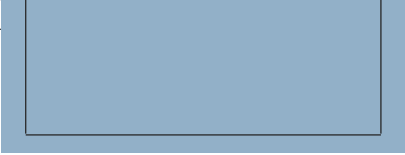
- In SVMs, gradient descent is used to find the optimal hyperplane that separates the data points into different classes with maximum margin.

Dimensionality Reduction:

- In techniques such as Principal Component Analysis (PCA), gradient descent is used to find the optimal eigenvectors that capture the maximum variance in the data.

Clustering:

- In clustering algorithms such as k-means, gradient descent is used to optimize the centroids of the clusters by minimizing the sum of squared distances between the data points and their assigned cluster centroids.



THANK YOU!!